Q10. Linear Regression

Original Aim: To build a Linear Regression model using the Iris dataset to predict PetalLengthCm. Use SepalLengthCm, SepalWidthCm and PetalWidthCm as input features, split the data into training and testing sets, and train the model. Display the model coefficients and predictions for the test data.

Aim

Predict PetalLengthCm from SepalLengthCm, SepalWidthCm and PetalWidthCm; show fitted coefficients, intercept and test predictions.

Requirements

Python, NumPy, Pandas and scikit-learn's splitting, LinearRegression and regression metrics. Reference: [D10].

Essential theory

Multiple linear regression estimates y_hat=b0+b1*x1+b2*x2+b3*x3 by minimizing training squared residuals. Each coefficient represents the fitted change in predicted petal length for a one-unit change in that feature while holding the other inputs fixed. This is an association in the fitted model, not a causal claim. The intercept is the prediction at zero for all inputs, which lies outside these flowers' measurement range and need not have useful biological meaning.

Step-by-step procedure

1. Select exactly the three assigned input features; select PetalLengthCm separately as y.
2. Split into 120 training rows and 30 testing rows with test_size=0.20 and random_state=42.
3. Instantiate LinearRegression and fit only X_train/y_train.
4. Predict for X_test; pair predictions with the retained y_test row indices.
5. Display coefficients, intercept and all 30 actual/predicted values with residuals.
6. Report MAE, RMSE and R2 as supporting held-out regression metrics.

Complete Python program

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)

inputs = ["SepalLengthCm", "SepalWidthCm", "PetalWidthCm"]
X = df[inputs]
y = df["PetalLengthCm"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
coefficients = pd.Series(model.coef_, index=inputs, name="Coefficient")
comparison = pd.DataFrame({"Actual": y_test, "Predicted": predictions})
comparison["Residual"] = comparison["Actual"] - comparison["Predicted"]
print("Train/test shapes:", X_train.shape, X_test.shape)
print("COEFFICIENTS")
print(coefficients.round(6).to_string())
print(f"Intercept: {model.intercept_:.6f}")
print("ALL TEST PREDICTIONS (original row index)")
print(comparison.round(4).to_string())
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
print(f"MAE={mae:.6f}; RMSE={rmse:.6f}; R2={r2:.6f}")
```

Code explanation

- `X` is a two-dimensional table; `y` is the numerical target Series. PetalLengthCm is deliberately excluded from X to prevent target leakage.
- `train_test_split` shuffles reproducibly; no stratification is needed for this continuous-target example.
- `fit` estimates coefficients/intercept from training rows; `predict` applies the fitted relationship to unseen test rows.
- Building comparison with y_test keeps original indices and prediction order together. Residual=actual-predicted; a negative residual means overprediction.
- MAE is mean absolute error; RMSE takes the square root of mean squared error; both have centimetre units. R2 compares squared errors with a constant test-mean baseline and can be negative on poor models.

Verified output

```
Train/test shapes: (120, 3) (30, 3)
COEFFICIENTS
SepalLengthCm    0.722815
SepalWidthCm    -0.635816
PetalWidthCm     1.467524
Intercept: -0.262196
```

Row	Actual	Predicted	Residual	Row	Actual	Predicted	Residual
73	4.7	4.1277	0.5723	56	4.7	4.5414	0.1586
18	1.7	1.8820	-0.1820	104	5.8	5.7572	0.0428
118	6.9	7.0257	-0.1257	69	3.9	3.8103	0.0897
78	4.5	4.4321	0.0679	55	4.5	3.9853	0.5147
76	4.8	4.9272	-0.1272	132	5.6	5.8121	-0.2121
31	1.5	2.0662	-0.5662	29	1.6	1.3939	0.2061
64	3.6	3.8495	-0.2495	127	4.9	4.8811	0.0189
141	5.1	6.1295	-1.0295	26	1.6	1.7771	-0.1771
68	4.5	5.0217	-0.5217	128	5.6	5.6653	-0.0653
82	3.9	3.9745	-0.0745	131	6.4	5.9670	0.4330
110	5.1	5.3365	-0.2365	145	5.2	6.0485	-0.8485
12	1.4	1.4466	-0.0466	108	5.8	5.6327	0.1673
36	1.3	1.7814	-0.4814	143	5.9	5.9936	-0.0936
9	1.5	1.4553	0.0447	45	1.4	1.7401	-0.3401
19	1.5	1.4483	0.0517	30	1.6	1.5298	0.0702

```
MAE=0.260518; RMSE=0.360578; R2=0.960329
```

All 30 held-out predictions are included above. Coefficients are printed to six decimals while predictions/residuals are shown to four; the metric calculations use unrounded predictions.

Observation

The fitted coefficient of PetalWidthCm is positive and the coefficient of SepalWidthCm is negative conditional on the other inputs. The test RMSE is about 0.361 cm and R2 about 0.9603 for this split. For row 141, the model predicts about 6.1295 cm when the actual value is 5.1 cm, showing that a strong aggregate fit still permits individual errors. Correlated features and omitted structure can affect coefficients and predictions.

Result

A reproducible three-input linear model was trained without using the target as a feature, and its coefficients, intercept and held-out predictions were obtained. These predictions estimate values; they are not expected to equal every actual measurement.

Common errors and troubleshooting

- Expected 2D array: select input columns with a list, not a single unwrapped Series.
- Inconsistent samples: split X/y in the same call to preserve pairing.
- Wrong feature order for a new prediction: supply a DataFrame with the same named input columns in the same order.
- Accuracy score used for regression: choose MAE/RMSE/R2, not classification accuracy.

Viva questions and answers

- What is the dependent variable here? PetalLengthCm.
- What does a coefficient mean? The fitted conditional change in predicted target for a unit input change.
- Why reserve test rows? To evaluate predictions on examples not used for fitting.
- Why are predictions imperfect? A linear model only approximates the relationship and measurements vary.